# On Verification & Validation of Neural Network Based Controllers

Johann Schumann[†], Pramod Gupta[‡] and Stacy Nelson[⋆]
[†] RIACS / NASA Ames, schumann@email.arc.nasa.gov
[‡] QSS / NASA Ames, pgupta@email.arc.nasa.gov
[⋆] Nelson Consulting Company, NelsonConsult@aol.com

**Abstract.** Artificial neural networks (ANN) are being used as an alternative to traditional models for a range of applications, e.g., prediction, pattern recognition, and process control. Unfortunately, ANN models rarely provide any indication of an accuracy or reliability of their predictions. Before ANNs can be used in safety critical applications (aircrafts, nuclear plants etc.), a certification process must be established for ANNs based controllers. Traditional approaches to validation of neural networks are mostly based on empirical evaluation through simulation and/or experimental testing. On-line trained ANN used in safety critical applications are subject to strict certification standards and traditional methods of verification and validation cannot be applied to them, leaving a wide technological gap - which we attempt to address in this paper. We consider the particular attributes of ANNs to arrive at a set of requirements for a standard by which they could be certified.

**1. Introduction.** During the recent past, the interest in research and application of universal model-free controllers has increased substantially. Considerable research has been done to use artificial neural networks for such controllers. A number of successful application areas in the realm of NASA has been identified: adaptive control of various types of manned and unmanned aircraft or space craft, space shuttle docking, or neural network based sensor failure detection, just to mention a few. In all cases, superior control behavior and high adaptability could be obtained together with drastically reduced development and operation costs. Practical feasibility of a neural network based controller for aircraft is currently being investigated at NASA for a manned F-15 Active aircraft based at Dryden Flight Research Center, as well as for a C-17 transport aircraft. Here, on-line trained neural networks within the flight controller can adapt to sudden damage or loss control surfaces, a situation likely to cause fatal accidents.

Actual use of neural networks in safety critical areas has been severely limited. A major reason for that is, that each piece of software in a safety critical application needs to undergo a software development process which calls out for extensive and rigorous *verification and validation* (V&V) of the software. Usually, a specific software *certification* procedure has to ensure that under all circumstances, the system works reliably and without failures. For example, all safety critical software for U.S. commercial avionics has to be certified by the FAA, according to the DO-178B standard.

The aim of this paper is to develop requirements, which could then be adapted to design a safety critical ANN certification process appropriate to a particular standard. In this paper, we will describe work done in the IFCS/V&V[1] project which develops methodology and software process for the V&V of neural network based aircraft controllers. We focus on two different ANN based controllers: one uses pre-trained neural network and other one uses on-line trained neural network. In pre-trained NN (PTNN) case, the network is trained off-line and its weights are frozen before deployment. In other case, the neural network is trained while the system is operating. A PTNN network is obviously a purely deterministic mapping, and it may be analyzed just as any other function. Hence, its verification should not be more difficult than of other well-accepted implementation of non-linear mappings, such as polynomials or look up tables. Yet, due to misconceptions surrounding neural networks, they have been treated with significant suspicion, and regarded as unsafe. One of the objectives of this paper is to clarify this issue and discuss methods to verify the performance of a network. In this paper, we present a software process guide which has been developed within the IFCS project and which has been specifically designed to aid V&V of PTNNs. In addition to the points mentioned above, we will also discuss issues of off-line

[1] IFCS (Intelligent Flight Control System) is a joint project between NASA DFRC, NASA Ames, ISR, and Boeing.

training (selection of training data, over-training, etc.) as well as numerical issues (e.g., scaling, condition numbers).

The main criticism against inclusion of neural networks in safety critical applications has been their non-determinism. In applications involving on-line training, it is indeed impossible to predict, in a deterministic sense, the future connection weights and thus, the form of the neural network mapping. In addition, it was argued that it is difficult, or even impossible, to assess the true performance of a trained networki due to their time-variant characteristics. These are obviously valid concerns which need to be addressed in cases involving on-line adaptation/learning and real-life training data. In this paper we describe the highlights of a process guide which is being developed and discuss advanced V&V techniques and tools which are important for handling the on-line adaptive case.

**Application: Intelligent Flight Control.** In this section we briefly describe the architecture of the IFCS and the neural network that is employed as a component of the flight control system. The goal of IFCS is to develop and demonstrate a flight control system that can efficiently identify aircraft stability and control characteristics using neural networks and utilize this information to optimize aircraft performance in both normal and (simulated) failure conditions. It aims at taking advantage of nonlinear mapping capabilities of neural networks to develop on-line a mapping of key parameters of the aircraft dynamics. The aircraft used to test IFCS has been highly modified from a standard F-15 configuration to include canard control surfaces, thrust vectoring nozzles, and a digital fly-by-wire flight control system.

Figure 1 shows a block diagram of the IFCS as used within the project and described in [6].

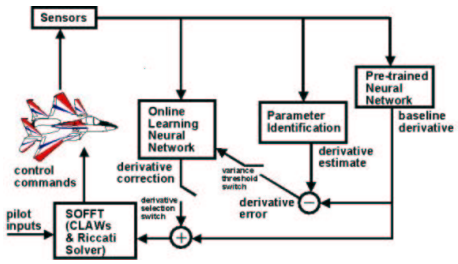



Figure 1: Overview of the IFCS architecture

For on-line adaptive configuration, both switches in Fig. 1 are closed. Thus actual sensor data from the PID are compared to the baseline derivatives. Deviations indicate a change of the aircraft's aerodynamic behavior which triggers the adaptation. These derivative errors are used to train the on-line neural network which actually learns to *correct* the data, produced by the PTNN. Thus, on-line neural network provides an adaptive mapping of the error between actual and baseline derivatives on the basis of the estimates produced during flight by the derivative estimator module and the baseline neural network mapping. The on-line mapping of the aerodynamic derivatives should guarantee improved performance since modeling errors and aging related variations are canceled out. Furthermore, the adaptive mapping should provide recovery capabilities in case of failure of system components by providing an estimate of the aerodynamic derivatives in the post-failure scenario.

**2. V&V of Neural Networks.** The main difference between ANNs and conventional implementations is that ANNs are trained by examples, taken from a set of training data representing the dynamics of the underlying system. For a conventional system to be certified, the specification is required to be unambiguous and, within reasonable bounds, complete. The conventional specification describes the behavior of the system for every circumstances, within which it is to operate. Ensuring that a system meets its specifications, is a central method of conventional certification

procedures. If this approach is to be preserved for ANN certification, the nature of the specification must be radically altered. A major pre-requisite to developing high-quality software is to use an appropriate *Software process* which covers all stages from initial requirements to deployment and which is specifically tailored toward adaptive NN based systems (explained in next section). Based on the properties of the development process for an ANNs, we can specify requirements for a certification standard: 1) How the goals or requirements for ANN are to be obtained, 2) what should be done to ensure that the training data adequately represent the system, 3) what type of networks can be used (i.e., number of nodes, number of layers, connectivity etc.), 4) what details the ANN developer must provide regarding the way in which the ANN module interfaces with the rest of the system and 5) what methods are to be used for quality assurance in the trained network.

When it comes to actually perform V&V on a NN-based system, the primary focus of the analysis is laid on the neural network specific tasks of training, learnability, convergence, and generalization. Quite often, however, an important aspect is overlooked: a neural networks is in essence a numerical method to approximate a function; its training method is usually an algorithm to solve an unconstrained quadratic optimization problem (minimization of the error $E = \sum_i (o_i - t_i)^2$ where $o_i$ is the actual NN output and $t_i$ the desired output). Therefore, a process for V&V of neural networks also needs to investigate the numerical issues underlying the NN and its implementation. Within this project, [12] discusses a list of important issues which need to be taken into account when V&V is attempted. Although the issues discussed in the following are tailored toward PTNN, principally the same issues occur when the neural network is trained on-line.

We propose a multi-layered approach to V & V of NN based systems (Figure 2). For the

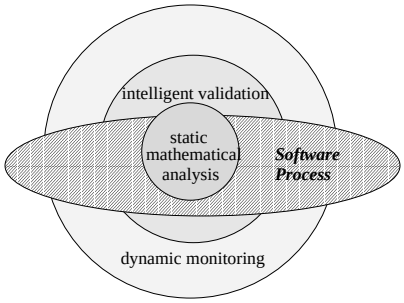



Figure 2: Layers of NN V&V methods

individual techniques, we identify three layers. The core-layer contains rigorous, mathematically sound results concerning robustness, stability, and convergence. Current state-of-the-art, however, only can provide relatively weak results, often in form of asymptotic guarantees. Typical examples here include Lyapunov proofs of (asymptotic) stability, or Vapnik-Cherenovis-dimension arguments to reason about the NN's generalization abilities.

However, for our safety-critical applications, these results don't suffice, as, for example, they cannot provide convergence guarantees in the required short period of time (usually on the order of a few seconds). Thus, methods on the second layer need to address these issues by *intelligent testing*. Here, techniques to reduce the number of required test cases in the nominal and pre-analyzed failure case can be applied. Analysis of numerical issues, as well as tests for convergence and robustness of the system belongs to that layer.

For truly adaptive systems, however, we still don't have a guarantee for performance (will there ever exist one?). Here, the third layer comes into play: methods in this layer will *dynamically monitor* the NN and its behavior. Although it ultimately cannot provide the guarantee, it at least can return dynamic information on how the NN currently behaves and if the current state of the system is recoverable (healable) at all. Research on these methods are currently performed [11].

## 0.1 A PTNN V&V Process

In order for a neural network to fly on spacecraft or aircraft, it must be certified. In this context, certification is the process of obtaining a certificate from NASA and/or the Federal Aviation Authority (FAA) to indicate conformance with airborne software standards. One of the major prerequisites for certification is that all activities during the entire software development and software life cycle are performed according to a detailed *software process*. A process guidebook defines all steps of the process and provides guidance on the required documents and activities (e.g., which kinds of tests have to be carried out). In the project, a process guide for the IFCS is being developed. It is based upon applicable existing standards and best practices and has been augmented to specifically deal with the Neural Networks aspects of the system. A process guide for on-line adaptation is currently in work and closely follows the line of this guidebook. There are several applicable NASA and industry V&V Standards for IFCS described in [7]: In addition to the NASA standards, RTCA DO-178B [4] contains guidance for determining that software aspects of airborne systems and equipment comply with airworthiness certification requirements. In order to ensure that verification techniques for neural networks meet these V&V standards, the following guidance is provided for integrating V&V of PTNN into the Software Life Cycle.

First, a well-defined process with discrete Software Life Cycle phases Fig. 3 (Figure 3) including documented work products (called deliverables) must be defined for the overall project. This process must include analysis procedures established to ensure correctness of deliverables and scheduled reviews of major product releases. V&V of neural networks need to be applied to all phases of this Software Life Cycle process. They may be used to enhance rather than replace traditional testing.

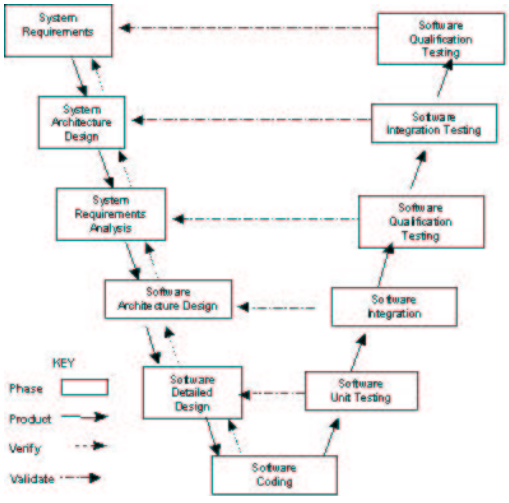



Figure 3: Lifecycle

In the following we list phases of the Life Cycle which have been substantially augmented for V&V of neural networks:

- *Systems Requirements*, *System Architectural Design*, and *Software Requirements Analysis*: Documentation for these phases must include the specification for the NN and its architecture. This description needs to contain the type of NN (feed-forward, Self Organizing Map, etc), the learning algorithm (gradient descent, Levenberg-Marquardt, Newton's method, etc.), and all parameters of the NN architecture (e.g., number of layers and number of nodes, initialization of weights). Furthermore, a concise description of the inputs and outputs (including units and

the expected and maximal range of values) and acceptable errors for PTNN must be provided. These design and requirements descriptions must be provided in detail, but not necessarily in a formal language. Software models (e.g., Simulink) are also acceptable; however all parameters which might be hidden in the simulator must be made explicit.

- *Software Detailed Design* must include a description of precise code constructs required to implement the NN, including all data structures and algorithms (e.g., libraries for matrix operations).
- *System Integration Testing* should verify that the architectural design is detailed enough so, when implemented, the NN can interface with system hardware and software in various fidelity testbeds.
- *System Qualification Testing* should verify that the system requirements are sufficient enough to ensure that, when implemented, the NN will interface properly with the system in production.

### 0.2 V&V Techniques for PTNN

# 1 V&V of on-line Adaptive Neural Networks

On-line adaptive systems in general, and learning neural networks in particular cannot be validated using traditional verification and validation techniques, because they evolve over time, and past learning data influences their behavior. While they hold great technological promise, on-line learning systems pose serious problems in terms of verification and validation, especially when viewed against the background of the tough verification standards that arise in their predominant application domains (flight control). Consider that methods for software product verification are generally classified into three categories: 1) Fault Avoidance, 2) Fault removal, and 3) Fault tolerance. Unfortunately, neither of these three methods is applicable as-is to adaptive systems, for the following reasons:

- *Fault Avoidance*: Formal design methods are based on the premise that we can determine the functional properties of a system by the way we design it and implement it. While this holds for traditional systems, it does not hold for adaptive systems, since their design determines how they learn, but not what they will learn.
- *Fault Removal*: Formal verification methods are all based on the premise that we can infer functional properties of a software product from an analysis of its source text. While this holds for traditional systems, it does not hold for adaptive systems, whose behavior is also determined by their learning theory. All testing techniques are based on the premise that the systems of interest will duplicate under filed usage the behavior that they have exhibited under test. While this is true for traditional deterministic systems, it is untrue for adaptive systems, since the behavior of these systems evolves over time.
- *Fault tolerance*: Fault tolerance techniques are based on the premise that we have clear expectations about the functions of programs and programs parts, and use these expectations to design error detection and error recovery capabilities. With adaptive systems, it is not possible to formulate such expectations because the functions of programs/program parts evolve.

### 1.1 V&V Techniques for OANN

## 2 Numerical Aspects of NN V&V

There are various issues related to the application of neural networks. Some of them are discussed below:

### 2.1 Choice of Input-Output Parameters

The preliminary step toward developing a neural network model is the choice/identification of the inputs and outputs of the problem to be modeled so that the corresponding training data can be generated. In general, the model outputs are determined based on the purpose of the model. Other factors influencing the choice of outputs are ease of data generation, ease of incorporation of the neural network model into the overall system and so forth.

### 2.2 Data Generation

Once the input(s)-output(s) have been identified, the next step in neural network development is the generation and collection of data for training the neural network. Initially a neural network does not have any information about the problem, the neural network has to be trained with the corresponding data. So it is important that the training data sufficiently represent the original problem. Training data must be provided to characterize the component to be modeled over a desired range of operation and for different combinations of geometrical and physical model inputs. The data must be collected with enough samples around stationary points and near curvature while a relatively sparse sampling of points in flat regions.

### 2.3 Scaling

Data scaling is an essential step to improve the learning process of neural networks. There are two major reasons to scale the training data. First, neural networks are limited to the range that their output can attain. For example, the common feed forward network with logistic activation function is theoretically limited to an output range between 0 and 1. Secondly, scaling gives equal importance to each input, and prevents premature saturation of the activation functions. The reason for this is that the inputs are usually combined as a weighted sum to produce the activation levels of the artificial neurons on the second layer. Thus, extremely large input values would either dominate the sum or would require very small weights, leading to low accuracy. A typical example for unfavorable ranges of data is when one input of the NN represents the current altitude in feet (range: $0 \ldots 50{,}000$ ft) and the other a small angle, e.g., $\alpha \in \{0, \ldots, 0.09\text{rad}\}$. In these cases, *scaling* of the inputs is important. Scaling can be performed on the input, output or both. Not so obvious, however, is the fact that scaling can affect the behavior of *some* training algorithms. For example, the well-known gradient descent algorithm is influenced by scaling, whereas a Newton method is not (for a detailed discussion see [3, 1, 5]).

### 2.4 Sensitivity/Conditioning

In order to assess the NN's behavior with respect to changes in the input values or the weights, *sensitivity analysis* can be performed. For a sensitivity analysis of the neural network during training, it is often helpful to consider, how small changes in the *weights* affect the error function (and thus how fast and good the network can be trained). We start from the the quadratic form of the error function $E$. Let $\Phi$ be a quadratic representation (or approximation) of the error function $E$ in the neighborhood of the (local) minimum $\mathbf{x}_0$. $\Phi$ is defined as

$$\Phi(\mathbf{x}) = \mathbf{g}^T\mathbf{x} + \frac{1}{2}\mathbf{x}^T\boldsymbol{G}\mathbf{x}$$

where $\mathbf{g} \in \mathbb{R}^n$ is the gradient $\nabla E|_{\mathbf{x}_0}$, and $\boldsymbol{G}$ is the Hessian matrix. When we plot $\Phi$ in the vicinity of the minimum, we obtain surfaces shaped like a bowl or an elongated valley; the contour lines are ellipses. Fig. 4 shows the contours for two examples. A training algorithm tries to find the minimum by moving small straight segments along this error surface. It is easy to see that such a search algorithm can find the minimum much easier in a round bowl than in a long and narrow valley where small deviations can cause the algorithm to get stuck or "thrown" out of the valley.

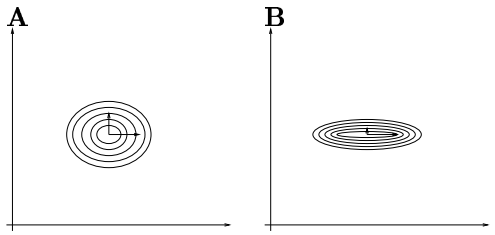



Figure 4: Contours of a well conditioned (A) and bad conditioned problem. The minimum is at the center of the ellipses, arrows mark the semi-axes of the ellipses.

More formally, the behavior of $\Phi$ (and thus also of $E$) in the vicinity of $\mathbf{x}_0$ is determined by the eigensystem of $\mathbf{G}$ The ratio of the largest and the smallest eigenvalue of $\mathbf{G}$ is defined as the condition number $\mathcal{C}$. $\mathcal{C}$ also determines the eccentricity of the ellipses in Figure 4: a small condition number corresponds to an almost circular error surface (A); a large $\mathcal{C}$ results in a steep and long "valley" (B) which can lead to serious problems. Thus, the condition number of the problem is an important metric for the behavior of the NN with respect to training and convergence. Thus, all tests which are performed during NN V&V should, in addition to demonstrating convergence, calculate and assess the condition number.

## 2.5 Training Algorithm

Training of a NN is an iterative numerical optimization algorithm, minimizing the error. Depending on the algorithm (e.g., gradient descent, Newton, Levenberg-Marqhart) and the training data, the algorithm must be stopped when optimal training of the NN has been reached. More specifically, the training algorithm needs to be stopped when: (i) the minimum has been reached (i.e., the problem has been solved), (ii) we have ground to a halt (i.e., there is no more progress), and (iii) we have exhausted memory or time resources.

In *all* cases, a careful analysis of the trained network and the reason why the training algorithm has stopped is important. [3, 5] elaborates on a number of possibilities why the training algorithm might have located a "wrong" local minimum or why it is oscillating or diverging, ultimately leading to an incorrect behavior of the NN-based system.

## 2.6 Confidence Interval/Estimation of Reliability

As pointed earlier that we are developing a layered approach for V & V of neural network based controller. Our second layer comprises a dynamic monitoring harness. Dynamic monitoring of the neural network can be seen as a safety net and protection harness which can guarantee that under no circumstances output values of the neural network can cause damage to the process and maneuver the aircraft into an unrecoverable state. Monitoring harness will check all input and output values of the neural network and thus can determine if the result of the neural network is reliable. The harness is based upon a statistical model of the system. This will enable us to produce a confidence measure on its output. This confidence measure is related to a novelty measure and allows to dynamically check the neural network's behavior: nominal conditions (or conditions seen

before) result in a high confidence value. In extreme cases of the parameter space (e.g., severe loss of control surfaces) error margins can and need to be much larger; resulting in a lower confidence. In that case, the behavior of the neural network will deviate from the ideal one (which might yield some roughness in handling), but it is still able to safely capture the modified behavior. These situations are fed into annunciator in teh control room display, which is activated when the neural network based controller is active for early warning. This display can be green (neural network has high confidence, failure/damage can be adapted easily), yellow (lower confidence), or red which indicates that the neural network has low probability to adapt to the present situation. This display produces very compact, and thus fast information for the pilot to facilitate decisions in critical situations.

Basically our method consist of calculating a confidence bound/interval on the output of the neural network. When ANN are used in prediction problems, it is usually desirable that some form of confidence interval is placed on the predicted value to measure the reliability of the neural. Confidence interval/bound is a measure of the precision of an estimated value. The interval represents the range of values, consistent with the data, that is believed to encompass the "true" value with high probability (usually 95%). Confidence interval provide a way to quantify confidence in the estimate of the neural network output. Wider intervals indicate lower precision, narrow intervals great precision (Fig. 5). It is shown that Bayesian techniques can be applied to assign confidence interval to the predicted output of the trained networks [[8], [9], [2], [10]]. The confidence value basically can be obtained if we donot only consider the output value of the ANN, but rather its probabilistic distribution, based on the distribution of the weights. The trained network can be descried by the posterior probability distribution of the weight values. Distribution of the weights give rise to distribution of the ANN's output [1]. The confidence level is in general closely related to the Hessian of the network. We are adapting and substantially extending techniques which allow to calculate these distributions, given the network and a statistical model of the data.

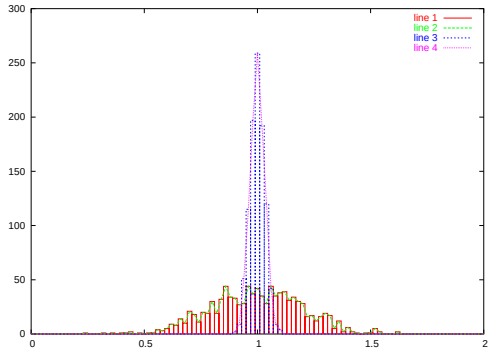



Figure 5: Variance

# 3 Conclusions

The V & V procedure discussed in the paper will allow the inclusion of neural network based controller in safety critical applications, and will pave the way for further progress in this area.

# References

[1] C. M. Bishop. *Neural Networks for Pattern Recognition.* Clarendon-Press, Oxford, 1995.

[2] W. Buntine and A. S. Weigend. Bayesian back-propagation. *Complex Systems*, 5:603–643, 1991.

[3] J. E. Dennis and R. B. Schnabel. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, volume 16 of *Classics in Applied Mathematics*. SIAM, 1996.

[4] Do-178b: Software considerations in airborne systems and equipment certification. URL: `http://www.rtca.org`, 1992.

[5] P. Gill, W. Murray, and M. Wright. *Practical Optimization*. Academic Press, 1981.

[6] C. Jorgensen. Direct adaptive aircraft control using neural networks. Technical Report TM-47136, NASA, 1997.

[7] D. Mackall, S. Nelson, and J. Schumann. Verification and validation of neural networks of aerospace applications. Technical Report TM-02-, NASA, 2002.

[8] D. MacKay. Bayesian interpolation. *Neural Computation*, 3(4):415–447, 1992a.

[9] D. MacKay. A practical bayesian framework for back-propagation networks. *Neural Computation*, 3(4):448–472, 1992b.

[10] R. M. Neal. *Bayesian Learning for Neural Networks*. PhD thesis, University of Toronto, Canada, 1994.

[11] J. Schumann. Vericonn: Verification of controllers based on adaptive neural networks — white paper—. Technical report, NASA Ames, Automated Software Engineering, 2001.

[12] J. Schumann. V&V issues for neural networks. Technical Report RIACS-TR-XX-02, RIACS, 2002.